

Versão: 2.0 | Autor: Ériston Vitor Fernandes Brito | Análise técnica do projeto

Projeto: Sistema de Gestão e Controle de Pedidos — Guanambusiness

1. Contexto: por que refatorar?

O “sgcp.py” original tinha **2.468 linhas em um único arquivo**. Tudo conexão com banco, lógica de negócio, autenticação, cálculos analíticos e toda a interface gráfica conviviam no mesmo espaço. Isso funcionou bem para construir o sistema, mas começa a criar problemas reais à medida que o projeto cresce:

- Qualquer alteração no banco exige mexer no mesmo arquivo que cuida dos botões
- Adicionar um relatório novo significa entender 2.000 linhas de contexto misturado
- Testar a lógica de autenticação sem abrir a janela gráfica é impossível
- Adaptar para mobile implicaria reescrever tudo do zero

A refatoração não mudou **o que o sistema faz**, mudou **como está organizado**, separando responsabilidades em camadas independentes.

2. Estrutura resultante

```
SGCP/
|
├── main.py          ← ponto de entrada único
|
├── app/
|   ├── config.py    ← constantes e caminhos (nenhuma lógica)
|   ├── state.py     ← estado mutável global (CARDAPIO, índice)
|   └── theme.py     ← temas, cores, HoverButton, brl()
|
|   ├── db/          ← CAMADA DE DADOS
|       ├── connection.py ← conexão MongoDB + helpers JSON
|       ├── pedidos.py  ← CRUD de pedidos
|       ├── cardapio.py ← CRUD do cardápio + normalização
|       └── usuarios.py ← CRUD de usuários + seed
|
|   ├── logic/       ← CAMADA DE NEGÓCIO (sem UI)
|       ├── auth.py   ← autenticação, perfis, cadastro
|       ├── cardapio.py ← índice, tempo médio, inserção
|       └── analytics.py ← fluxo, vendas, mesas, resumo do dia
|
|   └── views/        ← CAMADA DE INTERFACE (Tkinter)
|       ├── base.py   ← janela, layout, menu, tick, navegação
|       ├── login.py  ← tela de login e logout
|       ├── dashboard.py ← home com métricas e gráficos
|       ├── pedidos.py ← lista, filtros, cards, novo pedido
|       ├── cardapio.py ← catálogo, ficha técnica, formulários
|       └── usuarios.py ← gerenciamento de contas (admin)
|
└── data/            ← dados locais (fora do controle de versão)
    ├── .env
    ├── usuarios.json
    ├── pedidos.json
    ├── cardapio.json
    └── theme.json
```

3. O que cada parte faz

“main.py” Ponto de entrada

O único lugar onde tudo se junta. Inicializa o banco, carrega o tema, monta a classe final por composição de mixins e abre a janela. Tem menos de 40 linhas. Qualquer outro ponto de entrada, um servidor de API, um app mobile ou um script de linha de comando poderia reaproveitar tudo exceto estas 40 linhas.

“app/config.py” constantes

Contém apenas valores fixos: caminhos de arquivo, nomes dos perfis (`admin`, `atendente`), total de mesas e versão do app. Nenhuma função, nenhuma lógica. Quando for necessário aumentar o número de mesas de 14 para 20, ou mudar o caminho de dados, o arquivo inteiro é editado em segundos.

“app/state.py” Estado compartilhado

Guarda o “CARDAPIO” carregado em memória e o índice rápido de itens. É um módulo simples com duas variáveis. Qualquer parte do sistema importa “import app.state as state” e acessa “state.CARDAPIO” diretamente. Isso evita passar a lista de cardápio como parâmetro por todas as funções e garante que todos estão olhando para os mesmos dados.

Camada “db/” — Persistência de dados

Esta camada tem uma única responsabilidade: **ler e gravar dados**, sem saber nada sobre a interface ou a lógica do negócio.

“**connection.py**” gerencia a conexão com o MongoDB. Lê o “data/.env”, tenta conectar, expõe “MONGO_OK” (bool) e “get_db()”. Se a conexão falhar, o sistema simplesmente usa JSON local — isso é o fallback. Nenhuma outra parte do código precisa se preocupar com isso; basta importar “from app.db.connection import MONGO_OK”.

“**pedidos.py**” contém exatamente três operações: “carregar()”, “salvar()” e “atualizar_campo()”. Internamente cada uma verifica “MONGO_OK” e usa MongoDB ou JSON conforme disponível. A lógica que chama essas funções não sabe e não precisa saber onde os dados estão.

“**cardapio.py**” faz o mesmo para o cardápio, com um extra importante: a função “normalizar()”. Ela garante que qualquer documento vindo do banco inclusive documentos criados manualmente no Compass com campos faltando seja corrigido antes de chegar na interface. Isso protege o sistema contra dados malformados sem precisar tratar erro em cada tela.

“**usuarios.py**” cuida das contas. Inclui a função “seed()” que cria usuários padrão tanto no MongoDB (na primeira conexão) quanto no JSON local (na primeira execução sem banco). A migração automática de JSON → MongoDB acontece aqui.

Camada “**logic/**” Regras de negócio

Esta é a camada mais valiosa da arquitetura. Contém toda a lógica que determina **o que o sistema faz**, completamente livre de interface gráfica. Não importa “tkinter” em nenhum lugar.

“**auth.py**” trata autenticação: “autenticar()” verifica email e senha com hash SHA-256; “is_admin()” consulta o perfil; “cadastrar()” válida e cria conta; “remover()” remove com as devidas proteções (ninguém remove a si mesmo). Estas funções poderiam ser chamadas por um endpoint de API REST futuramente sem nenhuma alteração.

“**cardapio.py**” (lógica) cuida do índice em memória e do cálculo de tempo médio. A função “calcular_tempo_medio()” recebe a lista de itens de um pedido, consulta o índice e retorna algo como “~22 min”. Toda a inteligência está aqui, a view só exibe o resultado. “inserir_em_categoria()” e “categoria_do_item()” também vivem aqui, operando sobre “state.CARDAPIO”.

“**analytics.py**” contém os cálculos dos gráficos: “calcular_fluxo()” agrupa pedidos por hora, “calcular_vendas()” soma o faturamento por hora, “mesas_ocupadas()” retorna o conjunto de mesas com pedidos ativos, e “resumo_dia()” retorna um dicionário com todos os indicadores de uma vez. O dashboard chama “resumo_dia(self.pedidos)” e recebe {"total": 8, "abertos": 3, "finalizados": 5, "faturamento": 412.50, "mesas_ocupadas": 3} pronto para exibir. Nenhum cálculo na view.

“**app/theme.py**” visual e componentes compartilhados

Contém os dois temas completos (‘_LIGHT’, ‘_DARK’), o dicionário “COLORS” que é atualizado in-place na troca de tema, a função “lerp_color()” para interpolação suave, o helper “brl()” de formatação monetária e a classe “HoverButton”.

A “HoverButton” é definida aqui porque é usada por “**todas**” as views. Se posteriormente for necessário adicionar um som de clique, ou mudar a velocidade da animação, uma única edição em “theme.py” afeta todos os botões do sistema.

Camada “views/” Interface gráfica

Cada arquivo é um **mixim**, uma classe Python com métodos, mas sem “__init__” próprio. Os mixins não herdam de “tk.Tk”, eles dependem de “self” existir como uma janela Tkinter, o que é garantido quando o “main.py” os compõe.

“**base.py**” tem a “BaseApp(tk.Tk)”: janela, loop de atualização (‘_tick’), método “navegar()”, “_montar_layout()” (que cria header + área scrollável + menu inferior) e “_montar_menu_inferior()”. É a espinha dorsal da interface. Qualquer feature que apareça em todas as telas (um badge de notificação, um botão de ajuda) é adicionada aqui.

“**login.py**” tem “LoginMixin” com “mostrar_login()”, “_validar_login()” e “_logout()”. A validação usa “from app.logic.auth import autenticar” que chama a lógica, mas não reimplementa nada.

“**dashboard.py**” tem “DashboardMixin” com a tela de home e todos os widgets de gráfico (‘_grafico_barras’, ‘_grafico_linha’, ‘_grafico_rosca’). Os dados já chegam prontos da camada de analytics; a view só desenha.

“**pedidos.py**” tem “PedidosMixin” cobrindo a lista de OS com filtro e busca, os cards individuais, o modal de detalhe, as ações de avançar status, desocupar mesa e criar novo pedido.

“**cardapio.py**” tem “CardapioMixin” com a tela de catálogo, ficha técnica, formulários de receita e categoria, todos com controle de acesso por perfil.

“**usuarios.py**” tem “UsuariosMixin” com o painel de gerenciamento de contas, visível apenas para administradores.

4. Como o main.py une tudo

```
python
from app.views.base    import BaseApp
from app.views.login   import LoginMixin
from app.views.dashboard import DashboardMixin
from app.views.pedidos import PedidosMixin
from app.views.cardapio import CardapioMixin
from app.views.usuarios import UsuariosMixin

class SGCPApp(LoginMixin, DashboardMixin, PedidosMixin,
               CardapioMixin, UsuariosMixin, BaseApp):
    def __init__(self):
        super().__init__()
        # inicialização do banco, tema, cardápio, pedidos...
        self.mostrar_login()
        self._tick()
if __name__ == "__main__":
    SGCPApp().mainloop()
```

Python resolve a herança múltipla pela MRO (Method Resolution Order): quando “self.mostrar_dashboard()” é chamado, o Python percorre os mixins na ordem declarada até encontrar o método. O resultado é exatamente igual ao arquivo monolítico, mas organizado.

5. Análise de escalabilidade e modularidade

O que ficou concretamente melhor:

Adicionar uma nova tela agora significa criar um arquivo em “views/”, escrever um mixin com os métodos da tela, e adicionar o mixin na lista de herança do `main.py`. Não há risco de quebrar o login ou os gráficos.

Adicionar um novo banco de dados (PostgreSQL, Firebase, SQLite) significa criar um novo arquivo em “db/” com as mesmas funções (`carregar`, `salvar`, `atualizar\_campo`) e uma nova flag. A camada de lógica e a interface não percebem a mudança.

Testar a lógica de autenticação é agora trivial:

```
python
from app.logic.auth import autenticar, cadastrar
# sem precisar abrir janela, sem Tkinter, sem banco
```

Adicionar uma API REST ao sistema significa criar um ponto de entrada que importe “app.logic” e “app.db” diretamente, reutilizando toda a lógica e persistência sem tocar na interface.

Trabalhar em equipe fica mais seguro: dois desenvolvedores podem trabalhar simultaneamente em “views/cardapio.py” e “app/logic/analytics.py” sem conflito de merge no Git, porque os arquivos são independentes.

O que não mudou

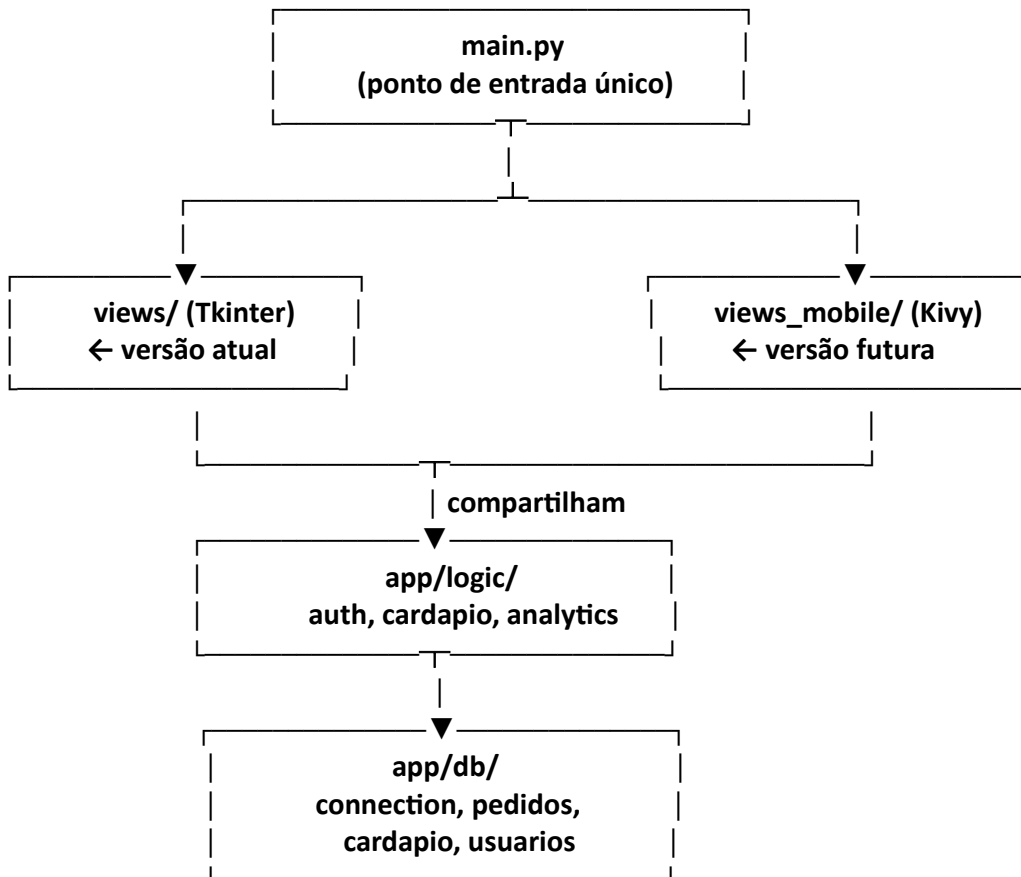
O comportamento do sistema é idêntico. Nenhuma funcionalidade foi removida ou alterada. O usuário final não percebe nenhuma diferença.

Onde ainda há espaço para evoluir

Os mixins de view ainda são relativamente longos. Em um projeto maior, cada modal (ficha técnica, formulário de receita, novo pedido) poderia ser extraído para seu próprio arquivo dentro de “views/”. A estrutura atual já suporta isso sem quebrar nada, é só mover o método para um novo mixin e adicioná-lo no “main.py”.

6. Caminho para mobile

A escolha de isolar completamente a UI na camada “views/” foi deliberada pensando na adaptação futura para dispositivos móveis.



Para portar para mobile com Kivy ou BeeWare, seria necessário reescrever apenas “app/views/” e adaptar “app/theme.py” (substituindo “HoverButton” por um widget Kivy equivalente). As camadas “db/” e “logic/” que contêm toda a inteligência do sistema seriam reutilizadas sem alteração.

Isso representa aproximadamente 60% do código total preservado em uma migração mobile, contra 0% se o sistema fosse reescrito a partir do monólito original.

7. Resumo comparativo

Aspecto:	Monólito (sgcp.py)	Modular (esta versão)
Linhas por arquivo:	2.468 (único)	50–250 por módulo
Adicionar nova tela:	Editar arquivo inteiro	Novo mixin isolado
Trocar banco de dados:	Refatoração ampla	Novo arquivo em “db/”
Testar lógica sem UI:	Impossível	Direto, sem dependências
Risco em cada mudança:	Alto (tudo no mesmo escopo)	Baixo (escopo isolado)
Trabalho simultâneo (Git):	Conflitos frequentes	Arquivos independentes
Portabilidade mobile:	Reescrita completa	Só a camada “views/”
Orientação para novo dev:	Difícil (sem fronteiras)	Clara (cada pasta tem papel)

8.Conclusão

A arquitetura modular é genuinamente melhor em escalabilidade e modularidade, não apenas no papel. Cada decisão de design tem uma consequência prática direta em como o projeto cresce e como uma eventual adaptação mobile pode ser feita com retrabalho mínimo.